Menu

Overview

Relevant source files

This document provides a comprehensive overview of the MKTY Smart Healthcare System, covering its architecture, core components, and integration patterns. The MKTY system is a distributed healthcare management platform that combines large language models (LLMs) and multimodal AI to provide intelligent diagnostic assistance, medical consultation, and healthcare workflow management.

For detailed information about specific subsystems, see System Architecture for technical implementation details, Frontend Application for user interface components, Backend Services for API endpoints and business logic, and AI & Machine Learning Services for AI model implementations.

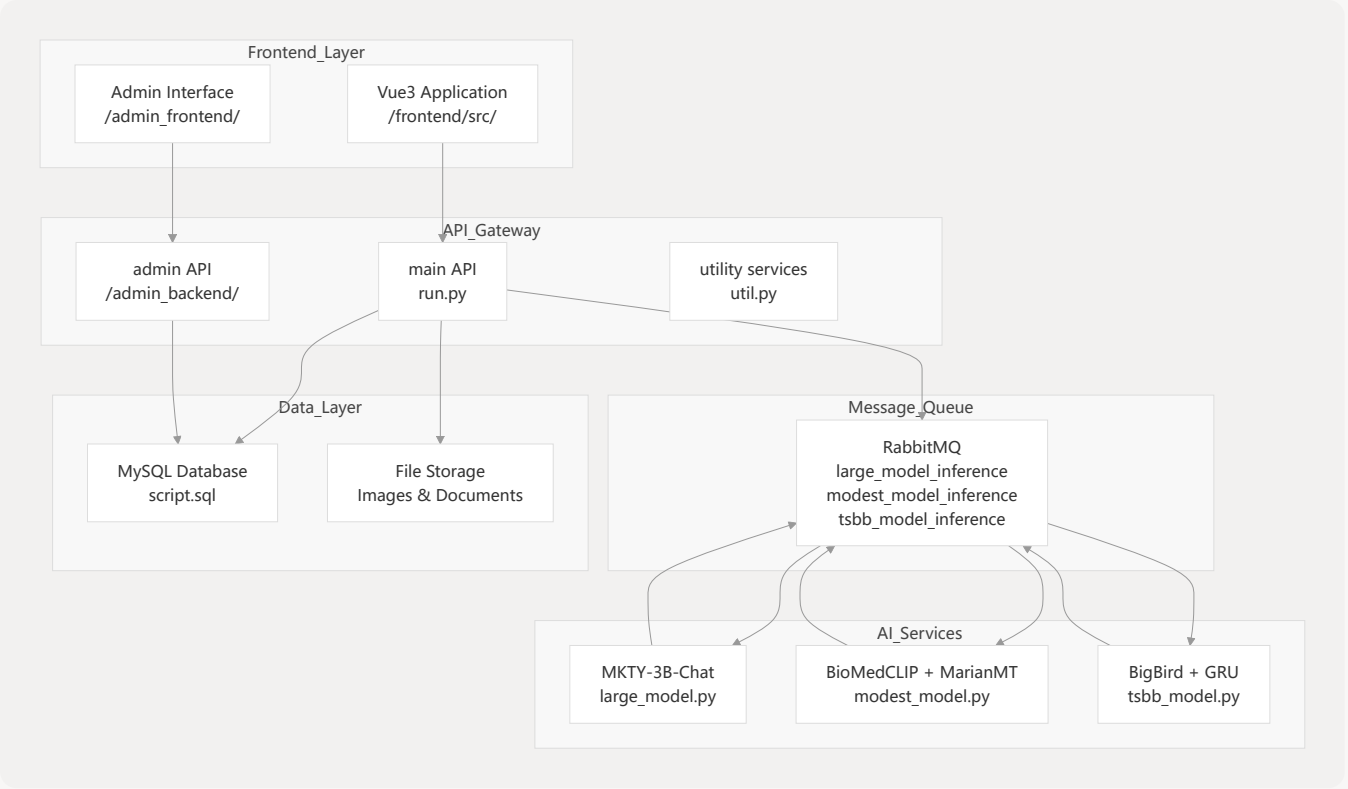
System Purpose and Scope

The MKTY Smart Healthcare System serves as an intelligent healthcare platform designed to enhance doctor-patient communication, streamline diagnostic workflows, and provide AI-powered medical assistance. The system integrates nine core functional modules through a distributed microservices architecture, supporting medical Q&A, multimodal diagnosis, forum discussions, and comprehensive medical record management.

Core Capabilities:

- AI-powered medical consultation using `MKTY-3B-Chat` language model
- Multimodal diagnostic assistance with `BioMedCLIP` and medical imaging analysis
- Community-driven medical forums with permission-based access control
- Comprehensive medical record and task management systems
- Administrative oversight and system monitoring capabilities

High-Level System Architecture



Sources: README.md 31-43 README.md 39-49

Core System Components

Frontend Applications

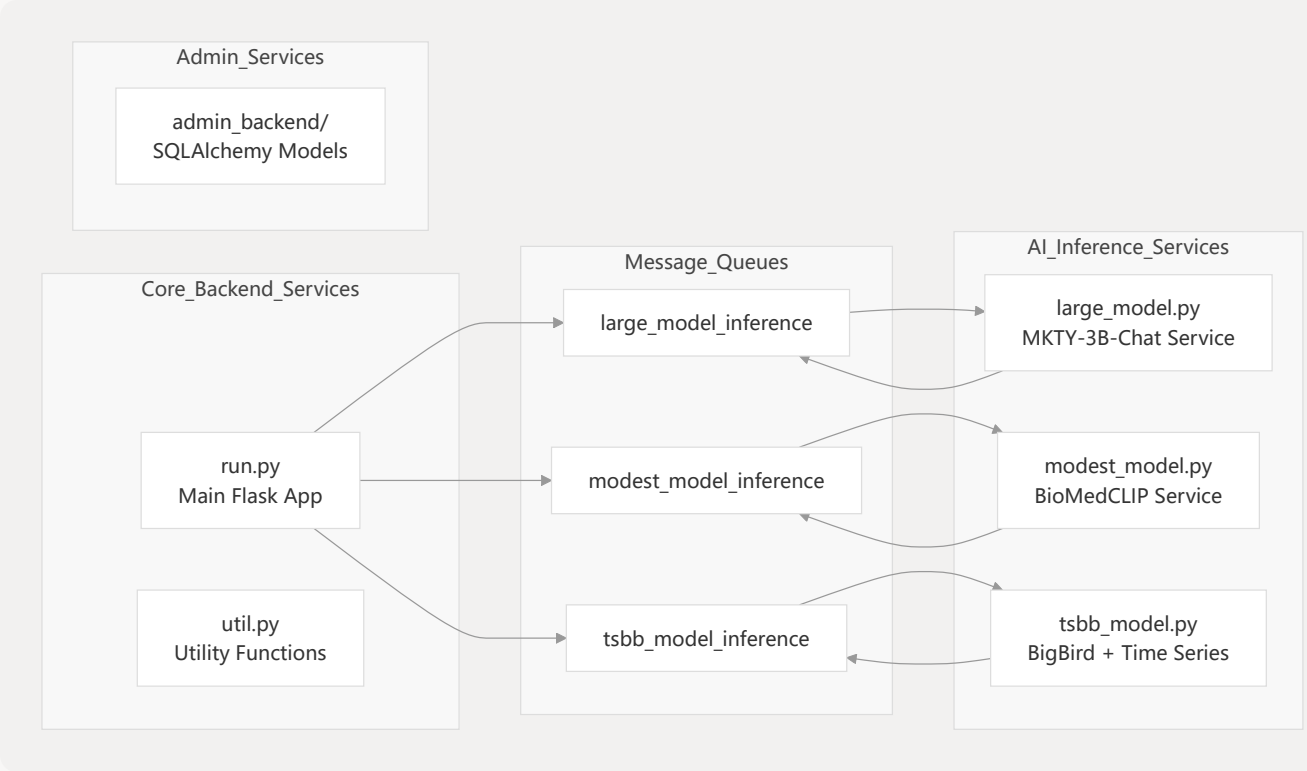
The system provides two primary user interfaces built with modern web technologies:

Component	Technology Stack	Primary Function
Main Frontend	Vue3 + Element Plus + Vite	Patient and healthcare provider interface
Admin Frontend	Vue + Vue-cli	Administrative dashboard and system management

Key Frontend Modules:

- **Authentication System**: User registration, login, and profile management
- **Medical Consultation Interface**: AI-powered Q&A and diagnostic tools
- **Forum Platform**: Community discussions with role-based permissions
- **Medical Records Management**: Patient history and treatment tracking
- **Task Management**: Important medical task lists and scheduling

Backend Services Architecture



Sources: `README.md` `350-394` backend file structure from table of contents

AI Services Integration

The system implements three specialized AI services that operate asynchronously through RabbitMQ message queues:

MKTY-3B-Chat Language Model Service

- **File Location**: `backend/large_model.py`
- **Queue**: `large_model_inference`
- **Capabilities**: Medical Q&A, treatment planning, diagnosis recommendations
- **Resource Requirements**: 8GB VRAM

Multimodal Diagnostic Service

- **File Location**: `backend/modest_model.py`
- **Queue**: `modest_model_inference`
- **Models**: `BioMedCLIP` for medical imaging + `MarianMT` for translation
- **Resource Requirements**: 2GB VRAM

Time Series and Text Analysis Service

- **File Location**: `backend/tsbb_model.py`
- **Queue**: `tsbb_model_inference`

- **Models**: `BigBird` for text embeddings + custom GRU for temporal prediction
- **Resource Requirements**: 2GB VRAM

API Endpoints and Data Flow

Sources: High-level diagrams provided, `README.md` `421-460`

Technology Stack Summary

Core Technologies

Layer	Technology	Purpose
Frontend	Vue3, Element Plus, Axios	User interface and API communication
Backend	Python Flask, JWT	Business logic and authentication
Database	MySQL 8.0+	Data persistence with JSON support
Message Queue	RabbitMQ	Asynchronous AI service communication
AI/ML	PyTorch, Transformers, OpenCLIP	Model inference and processing

Deployment Architecture

The system supports distributed deployment across multiple servers based on performance requirements:

- **Business Logic Tier**: Standard server requirements for Flask application
- **Database Tier**: MySQL 8.0+ with JSON data type support
- **AI Services Tier**: GPU-enabled servers with VRAM requirements:
  - MKTY-3B-Chat: 8GB VRAM
  - BioMedCLIP: 2GB VRAM
  - BigBird: 2GB VRAM

Development and Build Tools

- **Frontend Build**: Vite bundler with Node.js v22.12.0+
- **Package Management**: yarn for frontend, pip for Python backends
- **Environment**: Python 3.9+ across all backend services